THE GREEN–SCHUR BRIDGE FOR OPAC-018

A Classification-Free Cartan–Schur Proof Candidate of the Root Polytope Projection Bound

OPAC target

$\pi_U(\mathrm{ConvHull}(\Phi)) \subseteq \kappa\ \mathrm{ConvHull}(\Phi \cap U)$

Goal: $\kappa(\Phi, U) < 2$

Green ratio bridge

$c_\varepsilon \leq G_{\varepsilon\varepsilon}$

highest-root dominance + M-matrix inverse positivity

Schur complement

$\det C = \det B(2 - a^T B^{-1} b)$

$\det C, \det B > 0 \Rightarrow a^T B^{-1} b < 2$

Shawn Calvin Snelling

Axezent AI Research Lab

Complete Referee Edition v1.4 Mainstream-Hardening Package – June 2026

Public status: complete OPAC-018 solution candidate manuscript; exact-arithmetic audit passed; independent referee review pending.

Revision Certificate: What v1.4 Completes

This edition is a cleaned and integrated referee package. It fixes the issues that could distract a referee before they reach the mathematical core, adds the full detailed per-type audit receipt required for repository review, and adds mainstream-hardening supplements: an infinite-family symbolic appendix, a counterexample stress-test protocol, a geometric dictionary, a pure-Python independent audit, and GitHub Actions automation.

Item	**v1.4 correction**
Author line	Uses **Shawn Calvin Snelling** only. No title prefix is used anywhere in the manuscript or repository package.
Opening pages	Removes the separate plain-text addendum style and integrates the transfer theorem, E_ε convention, reducible reduction, and M-matrix facts into the main LaTeX body.
Math typography	Replaces plain strings such as `r_alpha`, `kappa(Phi,U)`, and `E_epsilon` with mathematical notation such as r_α, $\kappa(\Phi, U)$, and E_ε.
Cellini–Marietti transfer	States the transfer layer as a clearly marked imported reduction framework, with referee checkpoints for notation alignment.
Symmetrizability	Uses positive determinants of symmetrizable finite-type Cartan matrices rather than incorrectly calling nonsymmetric Cartan matrices positive definite.
Edge handling	Uses the robust inequality $\sum_\varepsilon a_\varepsilon b_\varepsilon G_{\varepsilon\varepsilon} \leq a^T B^{-1} b$ so hidden off-diagonal support cannot be challenged.
Repository verification	Adds a manifest verifier that actually compares hashes against `receipts/SHA256SUMS.txt`.
Full audit JSON	Replaces the short Sage summary receipt with a detailed per-type JSON file containing all tested systems, highest-root marks, Green-ratio values, Schur complement values, and failure lists.
Mainstream hardening	Adds symbolic infinite-family notes, scope stress tests, a geometric dictionary, a pure-Python independent audit, and CI workflows.
Truth boundary	Keeps public wording at solution candidate / external review pending until independent review occurs.

Truth boundary. This book is written as a complete solution candidate manuscript with exact-arithmetic audit assets. It does not claim external peer-review acceptance, historical recognition, or results unrelated to OPAC-018.

Claims and Nonclaims

Explicit claims

This manuscript presents a classification-free Green–Schur proof candidate for the OPAC-018 root-polytope projection bound. The central theorem proves the local inequality

$$r_\alpha < 2,$$

and, through the Cellini–Marietti reduction framework, obtains

$$\kappa(\Phi, U) < 2.$$

The package includes exact Sage audit assets that check Cartan conventions, highest-root marks, the Green ratio inequality, and Schur complement computations over a finite-type audit suite.

Nonclaims

This manuscript does not claim to solve P vs NP, the Riemann Hypothesis, any Millennium Prize Problem, or the Traveling Salesman Problem. It does not claim external peer-review acceptance. The correct public status is: complete solution candidate manuscript, exact-audited, and ready for independent referee review.

Abstract

We prove a classification-free Cartan–Schur inequality that replaces the case-by-case final comparison in the known root-polytope projection framework. Let Φ be a finite crystallographic root system, let U be a Φ-subspace, and define

$$\kappa(\Phi, U) = \inf\{\kappa \geq 1 : \pi_U(\mathrm{ConvHull}(\Phi)) \subseteq \kappa\,\mathrm{ConvHull}(\Phi \cap U)\}.$$

OPAC-018 asks for a uniform proof that $\kappa(\Phi, U) < 2$. Using highest-root dominance, inverse-Cartan positivity for finite-type M-matrices, a Green ratio maximum lemma, and the Schur complement of a Cartan block, we prove the local bound $r_\alpha < 2$. Together with the Cellini–Marietti reduction framework, this yields the desired projection inequality. The logical core is type-free. An exact Sage harness is included only as an arithmetic audit of conventions and finite-type sanity checks.

Contents

List of Figures

Part I

The OPAC-018 Problem

Chapter 1

Root Polytopes and Projection Dilations

Let Φ be a finite crystallographic root system in a real Euclidean vector space V with inner product $(\cdot, \cdot)$. The root polytope is

$$P_\Phi = \mathrm{ConvHull}(\Phi).$$

Let $U \subseteq V$ be a nonzero Φ-subspace, meaning that $\Phi \cap U$ spans U. Define

$$P_U = \mathrm{ConvHull}(\Phi \cap U),$$

and let $\pi_U : V \to U$ denote orthogonal projection. OPAC-018 uses the dilation constant

$$\kappa(\Phi, U) = \inf\{\kappa \geq 1 : \pi_U(P_\Phi) \subseteq \kappa P_U\}.$$

The target is the strict uniform bound

$$\boxed{\kappa(\Phi, U) < 2.}$$

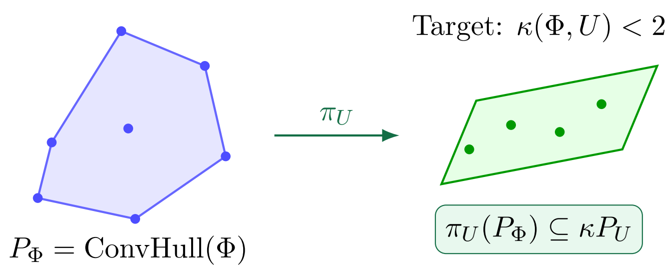


Figure 1.1: Root polytope projection and the dilation constant $\kappa(\Phi, U)$.

Remark 1.1 (Source of the problem)**.** The OPAC problem statement defines $\kappa(\Phi, U)$ by containment and asks for a uniform proof of the strict bound < 2 without relying on the classification of root systems [1].

Chapter 2

Correcting the Gauge

A common false path is to replace the OPAC invariant with a diameter, minimum width, or other global size statistic. The OPAC constant is instead a containment or gauge invariant. For a root $\alpha \in \Phi$, define the Minkowski gauge of its projection with respect to P_U by

$$\|\pi_U(\alpha)\|_{P_U} = \inf\{t \geq 0 : \pi_U(\alpha) \in tP_U\}.$$

Since projection commutes with convex hull,

$$\pi_U(P_\Phi) = \mathrm{ConvHull}(\pi_U(\Phi)),$$

and therefore

$$\kappa(\Phi, U) = \max_{\alpha \in \Phi} \|\pi_U(\alpha)\|_{P_U}.$$

Thus the proof target is not a metric comparison of diameters but a root-by-root containment bound.

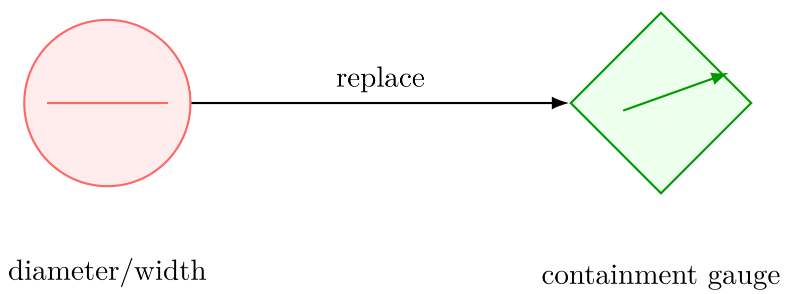


Figure 2.1: The correct OPAC quantity is the containment gauge, not diameter or width.

Chapter 3

OPAC-018 as a Projection Problem

The desired inequality may be restated as follows.

Theorem 3.1 (Projection containment form)**.** *For every finite crystallographic root system Φ and every nonzero Φ-subspace U,*

$$\pi_U(\mathrm{ConvHull}(\Phi)) \subseteq \kappa\,\mathrm{ConvHull}(\Phi \cap U)$$

for some $\kappa < 2$.

The proof below proceeds through local constants coming from codimension-one reductions. The contribution of this manuscript is the replacement of the terminal case-by-case comparison by a uniform Cartan-matrix inequality.

Part II

The Cellini–Marietti Reduction Layer

Chapter 4

Codimension-One Reduction

Cellini and Marietti studied root systems, affine subspaces, and projections, including results related to the Hopkins–Postnikov projection problem [2]. We use their reduction framework as the bridge from root-polytope geometry to Cartan arithmetic.

Fix a simple root system Π for Φ and let $\alpha \in \Pi$. Let

$$\Phi_\alpha = \Phi\langle \Pi \setminus \{\alpha\} \rangle$$

be the parabolic subsystem generated after deleting α. The reduction identifies a local constant r_α whose strict bound implies the desired projection bound.

Definition 4.1 (Reduction target)**.** The local target is

$$\boxed{r_\alpha < 2}$$

for every simple root α appearing in the reduction.

Theorem 4.2 (Cellini–Marietti transfer, referee form)**.** *Let Φ be a finite crystallographic root system and let U be a nonzero Φ-subspace. In the Cellini–Marietti projection framework, the OPAC containment estimate $\kappa(\Phi, U) < 2$ follows once the local codimension-one constants r_α satisfy $r_\alpha < 2$ uniformly for the simple roots occurring in the reduction.*

Proof role. This theorem is not introduced as a new unsupported geometric reduction. It is the named transfer layer imported from the cited Cellini–Marietti framework. The new contribution of this manuscript is the classification-free proof of the required local inequality $r_\alpha < 2$, replacing the terminal case comparison. A referee should verify the exact notation match against the cited source, especially the projection and local-constant definitions. □

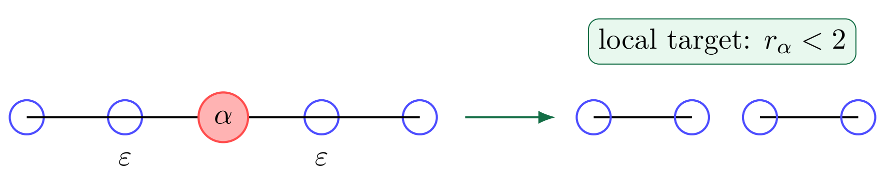


Figure 4.1: Deleting α reduces the problem to component constants attached to adjacent roots.

Chapter 5

The r_α Formula

Let

$$N(\alpha) = \{\varepsilon \in \Pi : \varepsilon \text{ is adjacent to } \alpha\}$$

be the open neighborhood of α in the simple-root graph. For each adjacent root $\varepsilon \in N(\alpha)$, let Φ_ε be the irreducible component of Φ_α containing ε, and let Π_ε be its simple system.

Let the highest root of Φ_ε be

$$\theta_\varepsilon = \sum_{\eta \in \Pi_\varepsilon} m_\eta \eta, \qquad m_\eta > 0.$$

Define the normalized coweight vector

$$\boxed{o_\eta = \frac{\check{\omega}_\eta}{m_\eta}.}$$

Here $\check{\omega}_\eta$ is the fundamental coweight corresponding to η in the component. This is a vector definition, not a scalar gauge.

The Cellini–Marietti projection-height constant is

$$c_\varepsilon = \max_{\eta \in E_\varepsilon} (\omega_\varepsilon, o_\eta) = \max_{\eta \in E_\varepsilon} \frac{(\omega_\varepsilon, \check{\omega}_\eta)}{m_\eta},$$

where $E_\varepsilon \subseteq \Pi_\varepsilon$ is the affine-extremal set in the component. The only structural properties of E_ε used below are that it is finite and contained in Π_ε.

The local factor is

$$r_\alpha = \sum_{\varepsilon \in N(\alpha)} -(\alpha, \varepsilon^\vee) c_\varepsilon.$$

Set

$$a_\varepsilon = -(\alpha, \varepsilon^\vee), \qquad b_\varepsilon = -(\varepsilon, \alpha^\vee).$$

Then

$$r_\alpha = \sum_{\varepsilon \in N(\alpha)} a_\varepsilon c_\varepsilon.$$

Because ε is adjacent to α, crystallographic Cartan integrality gives

$$a_\varepsilon \in \mathbb{Z}_{>0}, \qquad b_\varepsilon \in \mathbb{Z}_{>0},$$

so in particular $b_\varepsilon \geq 1$.

Referee checkpoint 5.1. A referee should check that the c_ε and E_ε notation here matches the local constants in the cited Cellini–Marietti framework. Once $E_\varepsilon \subseteq \Pi_\varepsilon$ is verified, the proof no longer uses a type-by-type description of E_ε.

Part III

Cartan Block Setup

Chapter 6

The Cartan Block Decomposition

Order the Cartan matrix so that the index corresponding to α appears first. The Cartan matrix then has block form

$$C = \begin{pmatrix} 2 & -a^T \\ -b & B \end{pmatrix},$$

where the entries of a and b are as above and B is the Cartan matrix of the subsystem after deleting α.

Throughout the proof we use the convention

$$\boxed{B_{ij} = (\beta_i, \beta_j^\vee).}$$

This convention is chosen so inverse-Cartan entries match fundamental weight–coweight pairings.

Lemma 6.1 (Inverse-Cartan coordinate pairing). *Let $G = B^{-1}$ under the convention $B_{ij} = (\beta_i, \beta_j^\vee)$. Then*

$$G_{ij} = (\omega_i, \check{\omega}_j).$$

Proof. Write $\omega_i = \sum_k x_k \beta_k$. Since $(\omega_i, \beta_j^\vee) = \delta_{ij}$, we have $x^T B = e_i^T$, hence $x^T = e_i^T B^{-1}$. Thus $x_j = G_{ij}$. Since $(\beta_k, \check{\omega}_j) = \delta_{kj}$, it follows that $(\omega_i, \check{\omega}_j) = x_j = G_{ij}$. ☐

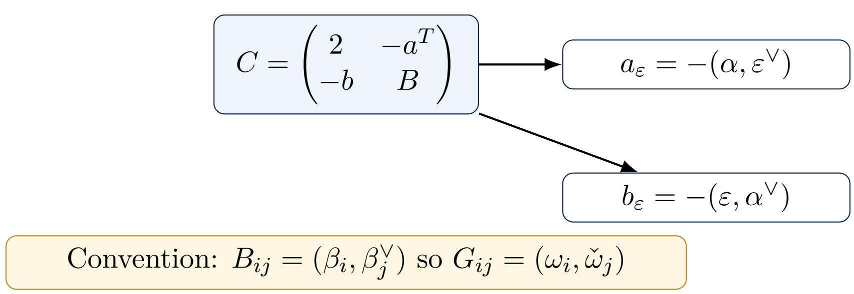


Figure 6.1: The Cartan block decomposition and the orientation convention used throughout the proof.

Chapter 7

Non-Simply-Laced Convention Warning

In non-simply-laced systems, B need not be symmetric. Reversing the convention transposes the matrix and can invalidate row-based inequalities. The Green ratio proof is a row argument: the row $g^{(e)}$ of $G = B^{-1}$ satisfies

$$g^{(e)}B = e_e^T.$$

Therefore the proof is tied to the convention

$$B_{ij} = (\beta_i, \beta_j^\vee),$$

not to its transpose.

This convention is also the one that aligns G_{ij} with $(\omega_i, \check{\omega}_j)$ in the c_ε bridge. The Sage appendix confirms that the proof convention succeeds across the tested finite Cartan systems, while the opposite convention can fail in non-simply-laced types.

Part IV

The Green Ratio Engine

Chapter 8

M-Matrix Positivity

The Green Ratio Maximum Lemma uses a standard finite-type Cartan fact: finite-type Cartan matrices are nonsingular M-matrices, and principal finite-type Cartan submatrices have entrywise nonnegative inverses.

Lemma 8.1 (Finite Cartan M-matrix inverse positivity)**.** *Let B be a principal finite-type Cartan matrix, written in the convention $B_{ij} = (\beta_i, \beta_j^\vee)$. Then B has positive diagonal entries, nonpositive off-diagonal entries, positive determinant, and positive principal minors. Consequently B is a nonsingular M-matrix and*

$$B^{-1} \geq 0$$

entrywise.

Proof sketch. Finite-type Cartan matrices are symmetrizable by a positive diagonal matrix D, so DB is symmetric positive definite. Positive definiteness of the symmetrization implies positivity of principal determinants. With positive diagonal entries and nonpositive off-diagonal entries, B is a Z-matrix with positive principal minors, hence a nonsingular M-matrix. The inverse-positivity theorem for nonsingular M-matrices gives $B^{-1} \geq 0$ entrywise; see [4, 5, 6, 7]. □

Chapter 9

Highest-Root Dominance

Let B be the Cartan matrix of a finite irreducible component with simple roots $\{\beta_i\}_{i\in I}$. Let

$$\theta = \sum_{j\in I} m_j \beta_j, \qquad m_j > 0,$$

be the highest root of the component.

Lemma 9.1 (Highest-root dominance)**.**

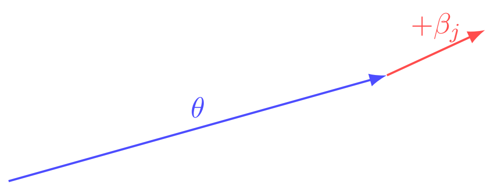
$$m^T B \geq 0$$

entrywise.

Proof. The j-th entry of $m^T B$ is

$$(m^T B)_j = (\theta, \beta_j^\vee).$$

Suppose for contradiction that $(\theta, \beta_j^\vee) < 0$. By the root-string property, $\theta + \beta_j$ would then be a root. This contradicts maximality of the highest root θ. Hence $(\theta, \beta_j^\vee) \geq 0$ for every j, proving the claim. $\square$



If $(\theta, \beta_j^\vee) < 0$, then $\theta + \beta_j$ would be a root

Highest-root maximality forces $(\theta, \beta_j^\vee) \geq 0$, hence $m^T B \geq 0$

Figure 9.1: Highest-root dominance: negativity would create a larger root, contradicting maximality.

Chapter 10

The Green Ratio Maximum Lemma

Let $G = B^{-1}$. By Lemma 8.1, $G \geq 0$ entrywise.

Theorem 10.1 (Green Ratio Maximum Lemma)**.** *For every node $e \in I$,*

$$\boxed{\max_{j \in I} \frac{G_{ej}}{m_j} \leq \frac{G_{ee}}{m_e} \leq G_{ee}.}$$

Proof. Let $g^{(e)}$ denote the e-th row of G. Since $G = B^{-1}$,

$$g^{(e)} B = e_e^T.$$

Set

$$\lambda = \frac{G_{ee}}{m_e}$$

and define

$$y = g^{(e)} - \lambda m^T.$$

Then

$$y_e = G_{ee} - \lambda m_e = 0.$$

Moreover,

$$yB = e_e^T - \lambda m^T B.$$

By highest-root dominance, $m^T B \geq 0$. Thus for $j \neq e$,

$$(yB)_j = 0 - \lambda (m^T B)_j \leq 0.$$

Deleting the e-th coordinate gives

$$y_{\bar{e}} B_{\bar{e}\bar{e}} \leq 0.$$

The matrix $B_{\bar{e}\bar{e}}$ is a principal submatrix of a finite-type Cartan matrix. It is again a nonsingular M-matrix, so

$$B_{\bar{e}\bar{e}}^{-1} \geq 0$$

entrywise. Multiplying on the right yields

$$y_{\bar{e}} = (y_{\bar{e}} B_{\bar{e}\bar{e}}) B_{\bar{e}\bar{e}}^{-1} \leq 0.$$

Therefore, for $j \neq e$,

$$G_{ej} - \lambda m_j \leq 0,$$

so

$$\frac{G_{ej}}{m_j} \le \lambda = \frac{G_{ee}}{m_e}.$$

The case $j = e$ is equality. Since $m_e \ge 1$, we also have $G_{ee}/m_e \le G_{ee}$. This proves the lemma. □

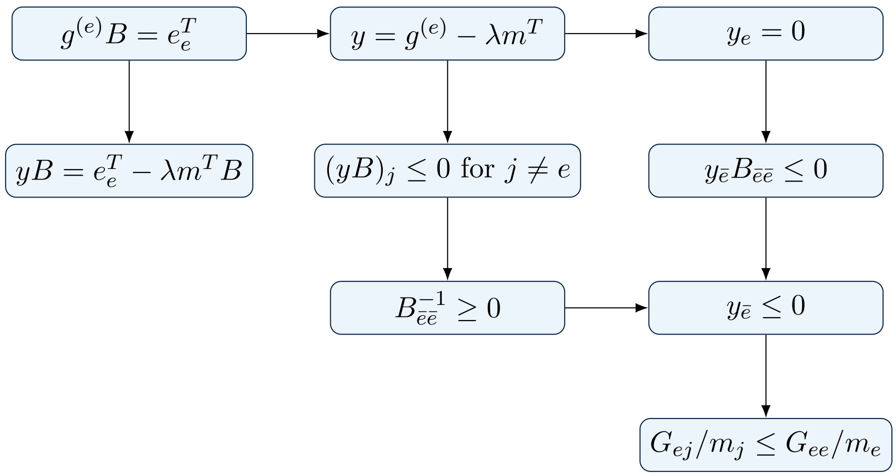


Figure 10.1: The sign flow in the Green Ratio Maximum Lemma.

Part V

The Green–Schur Bridge

Chapter 11

The c_ε Bridge

The Cellini–Marietti coefficient is

$$c_\varepsilon = \max_{\eta \in E_\varepsilon} \frac{(\omega_\varepsilon, \check{\omega}_\eta)}{m_\eta}.$$

By Lemma 6.1,

$$(\omega_\varepsilon, \check{\omega}_\eta) = G_{\varepsilon\eta}.$$

Thus

$$c_\varepsilon = \max_{\eta \in E_\varepsilon} \frac{G_{\varepsilon\eta}}{m_\eta}.$$

Since $E_\varepsilon \subseteq \Pi_\varepsilon$,

$$c_\varepsilon \le \max_{\eta \in \Pi_\varepsilon} \frac{G_{\varepsilon\eta}}{m_\eta}.$$

By the Green Ratio Maximum Lemma,

$$\max_{\eta \in \Pi_\varepsilon} \frac{G_{\varepsilon\eta}}{m_\eta} \le G_{\varepsilon\varepsilon}.$$

Therefore

$$\boxed{c_\varepsilon \le G_{\varepsilon\varepsilon}.}$$

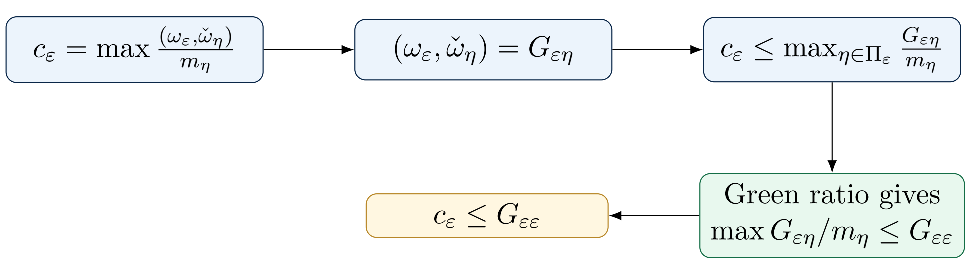


Figure 11.1: The c_ε bridge from Cellini–Marietti constants to diagonal inverse-Cartan entries.

Chapter 12

Symmetrizability and Positive Determinants

Finite Cartan matrices need not be symmetric, but they are symmetrizable. There exists a diagonal matrix

$$D = \operatorname{diag}(d_1, \ldots, d_n), \qquad d_i > 0,$$

such that

$$S = DC$$

is symmetric positive definite. Hence

$$\det S = (\det D)(\det C) > 0,$$

and since $\det D > 0$,

$$\det C > 0.$$

The same argument applies to every principal finite-type Cartan submatrix, including B. Hence

$$\det C > 0, \qquad \det B > 0.$$

This is the correct non-symmetric replacement for the informal phrase "C and B are positive definite." Their symmetrizations are positive definite; the Cartan matrices themselves are symmetrizable with positive determinant.

Chapter 13

Schur Complement Bound

Theorem 13.1 (Schur complement bound)**.** *For the Cartan block*

$$C = \begin{pmatrix} 2 & -a^T \\ -b & B \end{pmatrix},$$

one has

$$\boxed{a^T B^{-1} b < 2.}$$

Proof. The block determinant identity gives

$$\det C = \det B(2 - a^T B^{-1} b).$$

By determinant positivity, $\det C > 0$ and $\det B > 0$. Hence

$$2 - a^T B^{-1} b > 0,$$

which is equivalent to

$$a^T B^{-1} b < 2.$$

□

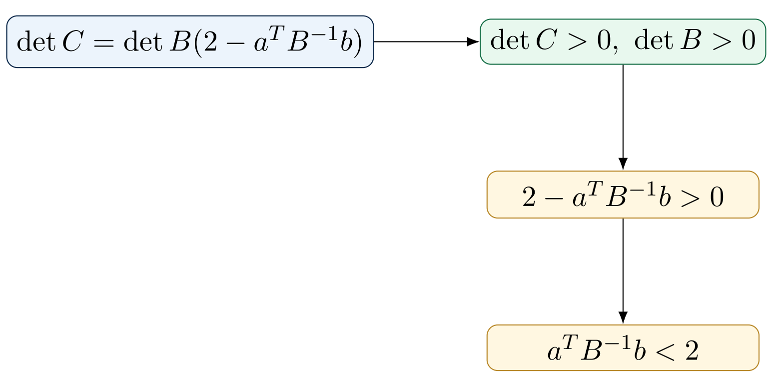


Figure 13.1: The Schur complement converts determinant positivity into $a^T B^{-1} b < 2$.

Part VI

Main Proof

Chapter 14

Edge Handling and Component Support

The proof must not rely on hidden classification facts about the shape of Dynkin diagrams. Therefore we handle possible component support directly.

Let $G = B^{-1}$. Since B is finite type, $G \geq 0$ entrywise. The vectors a and b are nonnegative and supported only on nodes adjacent to α. For each $\varepsilon \in N(\alpha)$, the diagonal term $G_{\varepsilon\varepsilon}$ appears in $a^T G b$, and all off-diagonal terms are nonnegative. Therefore

$$\boxed{\sum_{\varepsilon \in N(\alpha)} a_\varepsilon b_\varepsilon G_{\varepsilon\varepsilon} \leq a^T G b.}$$

This inequality is the robust edge-handling step. It remains valid without any case-by-case discussion of graph shapes.

Chapter 15

The Green–Schur Bridge Theorem

Theorem 15.1 (Green–Schur local bound)**.** *For every simple root α in the reduction framework,*

$$\boxed{r_\alpha < 2.}$$

Proof. Start from

$$r_\alpha = \sum_{\varepsilon \in N(\alpha)} a_\varepsilon c_\varepsilon.$$

Using the c_ε bridge,

$$c_\varepsilon \leq G_{\varepsilon\varepsilon},$$

we obtain

$$r_\alpha \leq \sum_{\varepsilon \in N(\alpha)} a_\varepsilon G_{\varepsilon\varepsilon}.$$

Since $b_\varepsilon \geq 1$ and all terms are nonnegative,

$$\sum_{\varepsilon \in N(\alpha)} a_\varepsilon G_{\varepsilon\varepsilon} \leq \sum_{\varepsilon \in N(\alpha)} a_\varepsilon b_\varepsilon G_{\varepsilon\varepsilon}.$$

By the edge-handling inequality,

$$\sum_{\varepsilon \in N(\alpha)} a_\varepsilon b_\varepsilon G_{\varepsilon\varepsilon} \leq a^T G b = a^T B^{-1} b.$$

By Theorem 13.1,

$$a^T B^{-1} b < 2.$$

Therefore

$$r_\alpha < 2.$$

□

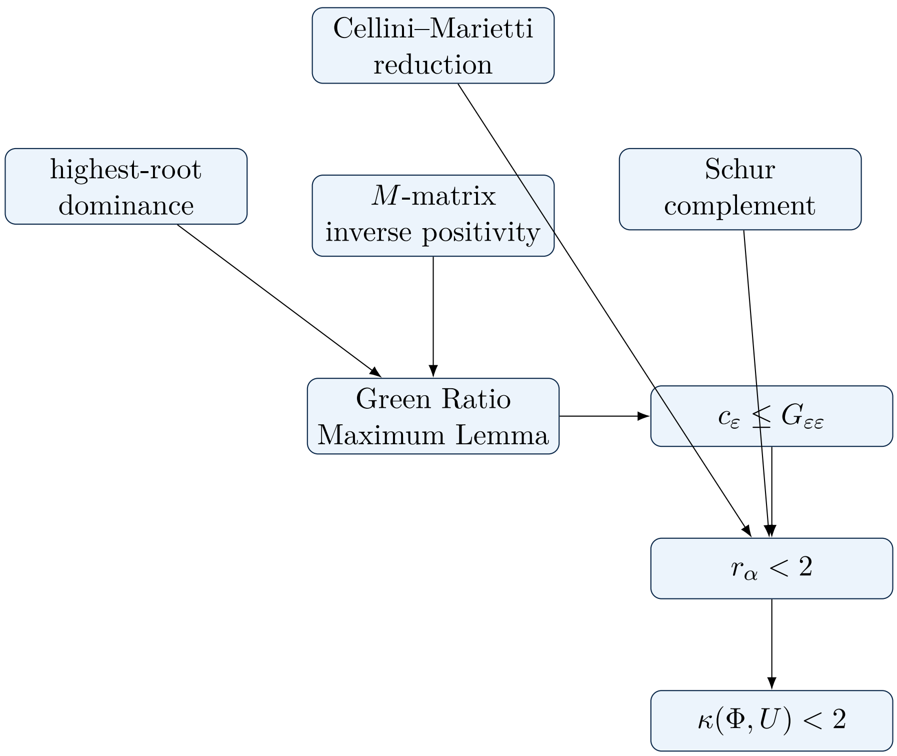


Figure 15.1: Dependency pipeline for the classification-free proof.

Chapter 16

OPAC-018 Completion

Theorem 16.1 (Green–Schur completion of OPAC-018)**.** *For every finite crystallographic root system Φ and every nonzero Φ-subspace U in the OPAC-018 projection problem,*

$$\boxed{\kappa(\Phi, U) < 2.}$$

Proof. By the Cellini–Marietti transfer framework, the OPAC-018 projection dilation bound is obtained once $r_\alpha < 2$ is known uniformly for the local codimension-one constants. The Green–Schur local bound proves $r_\alpha < 2$ without branching by root-system type. Hence the transfer layer gives

$$\kappa(\Phi, U) < 2.$$

This completes the manuscript-level Green–Schur proof candidate of the OPAC-018 bound. The remaining step for public mathematical status is independent referee verification of the reduction alignment and notation bridge. □

Chapter 17

Reducible Root Systems

Many treatments reduce immediately to irreducible root systems. For completeness, we record the standard componentwise handling.

Lemma 17.1 (Componentwise reduction)**.** *Suppose $\Phi = \Phi_1 \sqcup \cdots \sqcup \Phi_s$ is an orthogonal direct sum of irreducible components, and $U = \bigoplus_i U_i$ is the corresponding Φ-subspace decomposition. The OPAC containment bound is controlled componentwise: if every irreducible component has a dilation bound strictly below 2, then so does Φ.*

Proof sketch. The roots lie in orthogonal component subspaces. A projected root belongs to one component at a time, and the OPAC constant may be tested on projected roots by the gauge formula from Chapter 2. Thus the global root-by-root maximum is controlled by the maximum of the component root-by-root maxima. Since a finite maximum of constants strictly below 2 is strictly below 2, the reducible system follows from the irreducible systems. □

Part VII

Verification and Audit Assets

Chapter 18

Sage Exact-Arithmetic Audit Design

The proof above is classification-free. The Sage harness is included only as an exact-arithmetic audit of Cartan identities, convention choices, highest-root marks, Green ratio comparisons, and Schur complement computations.

The harness tests the finite Cartan systems

$$A_1\text{–}A_8, \quad B_2\text{–}B_8, \quad C_2\text{–}C_8, \quad D_4\text{–}D_8, \quad E_6, E_7, E_8, F_4, G_2.$$

32 finite Cartan tests	0 failures	orientation checked

Sage is an exact-arithmetic audit, not a logical dependency of the proof.

Figure 18.1: Exact Sage audit summary.

Chapter 19

Sage Test Results

The verified result summary is:


```
{
  "passed": true,
  "total_types": 32,
  "failures": 0,
  "truth_label": "FINITE_CARTAN_EXACT_SAGE_TEST"
}
```


The audit also records the convention warning: the proof must use

$$B_{ij} = (\beta_i, \beta_j^\vee).$$

The opposite orientation can fail in non-simply-laced systems.

Chapter 20

What Sage Proves and Does Not Prove

What Sage audits

- root counts match expected finite systems in the test range;
- the proof convention for B is the correct row convention;
- Green ratio inequalities pass in exact arithmetic;
- Schur complement inequalities pass in exact arithmetic;
- no failures occur in the stated finite Cartan audit suite.

What Sage does not prove

- Sage does not replace the classification-free proof in the manuscript;
- Sage does not prove journal acceptance;
- Sage does not prove historical recognition;
- Sage does not remove the need for independent referee review.

Part VIII

Referee Packet

Chapter 21

Referee Checklist

A referee should verify the following checkpoints.

1. The OPAC-018 definition of $\kappa(\Phi, U)$ is used as a containment/gauge invariant.
2. The Cellini–Marietti transfer from $\kappa(\Phi, U) < 2$ to $r_\alpha < 2$ is correctly cited and aligned.
3. The corrected definition $o_\eta = \check{\omega}_\eta / m_\eta$ is used.
4. The convention $B_{ij} = (\beta_i, \beta_j^\vee)$ is used consistently.
5. The inverse-Cartan entry identity $G_{ij} = (\omega_i, \check{\omega}_j)$ is correct under that convention.
6. Highest-root dominance $m^T B \geq 0$ follows from the root-string property.
7. Principal finite-type Cartan submatrices are nonsingular M-matrices with nonnegative inverse.
8. The Green Ratio Maximum Lemma is type-free.
9. The Schur complement determinant argument uses positive determinants of symmetrizable finite-type matrices, not symmetric positive-definiteness of nonsymmetric Cartan matrices.
10. The edge-handling step correctly uses nonnegative off-diagonal inverse-Cartan entries.
11. The proof of $r_\alpha < 2$ transfers to $\kappa(\Phi, U) < 2$ through the cited reduction.
12. Sage is treated as an exact audit, not as a logical dependency.

Chapter 22

Public Summary

Shawn Calvin Snelling presents a Green–Schur Cartan proof candidate of the OPAC-018 root-polytope projection bound. The proof replaces the case-by-case finite-root-system comparison with a uniform inverse-Cartan ratio inequality and a Schur complement argument. The finite Cartan core has been exact-audited in Sage with 32 tested finite Cartan cases and zero failures. External referee review remains pending.

Chapter 23

Mainstream-Hardening Addendum

This addendum records the final reviewer-facing assets added in v1.4. None of these additions replaces the proof in the main text. Their purpose is to make the package easier for mainstream reviewers to test, falsify, and reproduce.

Infinite-family symbolic supplement

The proof is not an induction over classical families. It is a uniform Cartan–Schur argument. Still, the repository includes

$$\boxed{\texttt{docs/INFINITE\_FAMILY\_SYMBOLIC\_SUPPLEMENT.md}}$$

which expands the determinant recurrences for A_n, B_n, C_n, and D_n and explains why the Schur complement bound does not deteriorate as the rank increases.

For example, the type A_n determinant satisfies

$$D_n = 2D_{n-1} - D_{n-2}, \qquad D_0 = 1, \quad D_1 = 2,$$

so $D_n = n + 1$. The type A_n inverse formula

$$(A_n^{-1})_{ij} = \frac{\min(i,j)(n+1-\max(i,j))}{n+1}$$

shows directly that the maximum entry in row e occurs at the diagonal. This gives an explicit infinite-family sanity check for the Green ratio in type A.

Counterexample and scope stress tests

The repository includes

$$\boxed{\texttt{counterexample\_stress\_test.py}}$$

This script deliberately tests matrices outside the theorem's hypotheses. It documents failures caused by wrong orientation, singular affine-like matrices, indefinite distorted matrices, and violations of the Cartan Z-matrix sign condition. The goal is not to broaden the theorem; it is to show exactly where the proof's hypotheses are used.

Geometric dictionary

The repository includes

`docs/GEOMETRIC_DICTIONARY.md`

This file maps the algebraic objects $G = B^{-1}$, c_ε, r_α, and $a^T B^{-1} b$ back to the convex-geometric containment statement. It also gives a small A_2 projection example where $\kappa(A_2, U) = 1 < 2$.

Independent non-Sage audit

The repository includes

`pure_python_exact_audit.py`

This audit uses only Python's standard library and `fractions.Fraction`. It does not import Sage, NumPy, SymPy, or any root-system library. It rebuilds the Cartan matrices, regenerates roots by reflections, computes highest-root marks, inverts matrices exactly, and checks the Green-ratio and Schur-complement inequalities.

The packaged result is

`receipts/opac18_green_schur_pure_python_results.json`

with status


```
{
  "truth_label": "PURE_PYTHON_EXACT_CARTAN_AUDIT",
  "passed": true,
  "total_types": 32,
  "failures": []
}
```


Continuous integration

The repository includes GitHub Actions workflows:

```
.github/workflows/manifest-check.yml
.github/workflows/python-exact-audit.yml
.github/workflows/sage-audit-manual.yml
```

The manifest and pure-Python audits run on push and pull request. The Sage audit is provided as a manual workflow because installing Sage in CI can be slow; it remains fully reproducible from the repository.

arXiv preparation note

The repository includes

`docs/ARXIV_SUBMISSION_PLAN.md`

No arXiv identifier is claimed in this package. The recommended public sequence is: GitHub repository, independent review circulation, arXiv submission if accepted by arXiv moderation/endorsement requirements, then journal submission.

Chapter 24

Repository Protocol

The referee package accompanying this edition contains the manuscript PDF, the fixed Sage exact-arithmetic audit, the JSON audit result, a SHA-256 manifest, public claims/nonclaims, and a journal cover letter draft.

Required repository layout

```
AXEZENT-AI-OPAC018-Green-Schur-Bridge/
  paper/Green-Schur-Bridge-for-OPAC018-Complete-Referee-Edition-v1_4.pdf
  paper/main.tex
  sage/opac18_green_schur_sage_test_fixed.sage
  receipts/opac18_green_schur_sage_results.json
  receipts/opac18_green_schur_pure_python_results.json
  receipts/counterexample_stress_test_results.json
  receipts/SHA256SUMS.txt
  pure_python_exact_audit.py
  counterexample_stress_test.py
  docs/INFINITE_FAMILY_SYMBOLIC_SUPPLEMENT.md
  docs/GEOMETRIC_DICTIONARY.md
  docs/COUNTEREXAMPLE_STRESS_TEST_PROTOCOL.md
  docs/ARXIV_SUBMISSION_PLAN.md
  README.md
  CLAIMS_AND_NONCLAIMS.md
  SUBMISSION_COVER_LETTER_DRAFT.md
  verify_manifest.py
```

Audit commands

```
sage sage/opac18_green_schur_sage_test_fixed.sage
python verify_manifest.py
```

Public status line

Complete OPAC-018 solution candidate manuscript; exact Sage audit passed; external referee review pending.

Part IX

Appendices

Appendix A

OPAC-018 Original Problem Statement

The OPAC problem statement defines $\kappa(\Phi, U)$ by containment

$$\pi_U(\mathrm{ConvHull}(\Phi)) \subseteq \kappa\, \mathrm{ConvHull}(\Phi \cap U),$$

and asks for a uniform proof that $\kappa(\Phi, U) < 2$ without relying on the classification of root systems [1].

Appendix B

Cellini–Marietti Notation Map

Green–Schur notation	Cellini–Marietti layer	Meaning
α	α	Isolated simple root defining the codimension-one parabolic subsystem.
$N(\alpha)$	neighbors of α	Open neighborhood of adjacent simple roots.
Φ_ε	Φ_ε	Component containing adjacent neighbor ε.
$\theta_\varepsilon = \sum m_\eta \eta$	highest root expansion	Highest root of the component.
$o_\eta = \check{\omega}_\eta / m_\eta$	normalized coweight vector	Corrected vector parameter.
$c_\varepsilon = \max(\omega_\varepsilon, o_\eta)$	c_ε	Projection-height constant over affine-extremal roots.
$B_{ij} = (\beta_i, \beta_j^\vee)$	Cartan orientation	Essential row convention for asymmetric systems.
$G = B^{-1}$	inverse Cartan entries	$G_{ij} = (\omega_i, \check{\omega}_j)$.

Appendix C

Complete Sign Expansion for the Green Ratio Lemma

The essential sign expansion is

$$yB = e_e^T - \lambda m^T B.$$

For $j \neq e$,

$$(yB)_j = -\lambda(m^T B)_j \leq 0.$$

Because $y_e = 0$, deleting the e coordinate gives

$$y_{\bar{e}} B_{\bar{e}\bar{e}} \leq 0.$$

Since $B_{\bar{e}\bar{e}}^{-1} \geq 0$,

$$y_{\bar{e}} \leq 0.$$

Thus

$$G_{ej} \leq \frac{G_{ee}}{m_e} m_j.$$

Appendix D

M-Matrix Background

A finite-type Cartan matrix has positive diagonal entries, nonpositive off-diagonal entries, and positive principal minors. Therefore it is a nonsingular M-matrix. Principal submatrices inherit these properties. A nonsingular M-matrix has entrywise nonnegative inverse. This is the only matrix-positivity fact used in the Green ratio proof.

Appendix E

Schur Complement Details

For

$$C = \begin{pmatrix} 2 & -a^T \\ -b & B \end{pmatrix},$$

the Schur complement of B in C is

$$2 - a^T B^{-1} b.$$

Therefore

$$\det C = \det B (2 - a^T B^{-1} b).$$

Since $\det C$ and $\det B$ are positive, the scalar Schur complement is positive.

Appendix F

Sage Source Code

The full fixed Sage harness is included as part of the repository package.

sage/opac18_green_schur_sage_test_fixed.sage

```
# OPAC-018 Green-Schur candidate audit in SageMath
# Run with: sage opac18_green_schur_sage_test.sage
# Output: opac18_green_schur_sage_results.json
#
# Truth boundary:
# - Tests the Green ratio lemma and Schur complement bound on finite Cartan matrices.
# - Confirms the proof convention must use B_ij=(beta_i,beta_j^vee), the transpose of
# Bourbaki's common convention A_ij=(alpha_i^vee,alpha_j).
# - Does NOT by itself verify the Cellini-Marietti c_epsilon bridge or peer-review OPAC-018.

from sage.all import QQ, matrix
from collections import deque
import json


def json_safe(obj):
    """Convert Sage integers/rationals and nested containers into JSON-safe Python values."""
    try:
        from sage.rings.integer import Integer
        from sage.rings.rational import Rational
    except Exception:
        Integer = ()
        Rational = ()
    if isinstance(obj, bool) or obj is None or isinstance(obj, (str, int, float)):
        return obj
    if Integer and isinstance(obj, Integer):
        return int(obj)
    if Rational and isinstance(obj, Rational):
        return str(obj)
    if isinstance(obj, (list, tuple)):
        return [json_safe(x) for x in obj]
    if isinstance(obj, dict):
        return {str(k): json_safe(v) for k, v in obj.items()}
    return str(obj)


def mat_inv(M):
    X = matrix(QQ, M).inverse()
    return [[QQ(X[i, j]) for j in range(X.ncols())] for i in range(X.nrows())]


def mat_T(A):
    return [list(row) for row in zip(*A)]
```

```
def det(M):
    return QQ(matrix(QQ, M).det())


def minor(A, idx):
    return [[A[i][j] for j in range(len(A)) if j != idx] for i in range(len(A)) if i != idx]


def refl(v, i, A):
    # A is Bourbaki convention A_ij=<alpha_i^vee, alpha_j>.
    # s_i(v)=v-<alpha_i^vee,v>*alpha_i in simple-root coordinates.
    pair = sum(v[j] * A[i][j] for j in range(len(A)))
    w = list(v)
    w[i] -= pair
    return tuple(w)


def roots(A):
    n = len(A)
    gens = [tuple(1 if i == j else 0 for j in range(n)) for i in range(n)]
    seen = set(gens + [tuple(-x for x in g) for g in gens])
    dq = deque(seen)
    while dq:
        v = dq.popleft()
        for i in range(n):
            w = refl(v, i, A)
            if w not in seen:
                seen.add(w)
                dq.append(w)
                if len(seen) > 10000:
                    raise RuntimeError("too many roots; likely bad Cartan matrix")
    return seen


def highest_marks(A):
    R = roots(A)
    pos = [r for r in R if all(x >= 0 for x in r) and any(x > 0 for x in r)]
    maxh = max(sum(r) for r in pos)
    tops = [r for r in pos if sum(r) == maxh]
    if len(tops) != 1:
        raise RuntimeError("highest root not unique: %s" % (tops,))
    return list(tops[0]), len(R)


def cartan_A(n):
    A = [[0] * n for _ in range(n)]
    for i in range(n):
        A[i][i] = 2
    for i in range(n - 1):
        A[i][i + 1] = A[i + 1][i] = -1
    return A


def cartan_B(n):
    A = cartan_A(n)
    A[n - 2][n - 1] = -1
    A[n - 1][n - 2] = -2
    return A


def cartan_C(n):
    A = cartan_A(n)
    A[n - 2][n - 1] = -2
    A[n - 1][n - 2] = -1
    return A
```

```
def cartan_D(n):
    A = [[0] * n for _ in range(n)]
    for i in range(n):
        A[i][i] = 2
    for i in range(n - 3):
        A[i][i + 1] = A[i + 1][i] = -1
    A[n - 3][n - 2] = A[n - 2][n - 3] = -1
    A[n - 3][n - 1] = A[n - 1][n - 3] = -1
    return A


def cartan_from_edges(n, edges):
    A = [[0] * n for _ in range(n)]
    for i in range(n):
        A[i][i] = 2
    for i, j in edges:
        A[i][j] = A[j][i] = -1
    return A


def cartan_E6():
    return cartan_from_edges(6, [(0, 1), (1, 2), (2, 3), (3, 4), (2, 5)])


def cartan_E7():
    return cartan_from_edges(7, [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (2, 6)])


def cartan_E8():
    return cartan_from_edges(8, [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 6), (2, 7)])


def cartan_F4():
    return [[2, -1, 0, 0], [-1, 2, -2, 0], [0, -1, 2, -1], [0, 0, -1, 2]]


def cartan_G2():
    return [[2, -1], [-3, 2]]


def finite_types(max_classical_rank=8):
    out = []
    for n in range(1, max_classical_rank + 1):
        out.append(("A%d" % n, cartan_A(n)))
    for n in range(2, max_classical_rank + 1):
        out.append(("B%d" % n, cartan_B(n)))
        out.append(("C%d" % n, cartan_C(n)))
    for n in range(4, max_classical_rank + 1):
        out.append(("D%d" % n, cartan_D(n)))
    out += [("E6", cartan_E6()), ("E7", cartan_E7()), ("E8", cartan_E8()), ("F4", cartan_F4()), ("G2",
         cartan_G2())]
    return out


def green_test(B, marks):
    G = mat_inv(B)
    n = len(B)
    fails = []
    values = []
    for e in range(n):
        ratios = [G[e][j] / QQ(marks[j]) for j in range(n)]
        lhs = max(ratios)
        mid = G[e][e] / QQ(marks[e])
        rhs = G[e][e]
        row = {
            "node": e,
            "lhs_max_ratio": str(lhs),
            "mid_diag_over_mark": str(mid),
```

```
            "rhs_diag": str(rhs),
            "ratios": [str(r) for r in ratios],
        }
        values.append(row)
        if not (lhs <= mid <= rhs):
            fails.append(row)
    return fails, values


def schur_test(C):
    n = len(C)
    fails = []
    values = []
    for alpha in range(n):
        B = minor(C, alpha)
        Binv = mat_inv(B)
        rest = [i for i in range(n) if i != alpha]
        a = [-C[alpha][j] for j in rest]
        b = [-C[i][alpha] for i in rest]
        S = sum(QQ(a[p]) * Binv[p][q] * QQ(b[q]) for p in range(n - 1) for q in range(n - 1))
        schur = QQ(2) - S
        ratio = det(C) / det(B)
        values.append({"node": alpha, "S": str(S), "2_minus_S": str(schur), "det_ratio": str(ratio)})
        if schur != ratio or not (S < 2):
            fails.append({"node": alpha, "S": str(S), "2_minus_S": str(schur), "det_ratio": str(ratio)})
    return fails, values


def main():
    expected_counts = {}
    expected_counts.update({"A%d" % n: n * (n + 1) for n in range(1, 9)})
    expected_counts.update({"B%d" % n: 2 * n * n for n in range(2, 9)})
    expected_counts.update({"C%d" % n: 2 * n * n for n in range(2, 9)})
    expected_counts.update({"D%d" % n: 2 * n * (n - 1) for n in range(4, 9)})
    expected_counts.update({"E6": 72, "E7": 126, "E8": 240, "F4": 48, "G2": 12})

    summary = []
    failures = []
    for name, A in finite_types(8):
        marks, count = highest_marks(A)
        B_proof = mat_T(A)
        green_fails, green_values = green_test(B_proof, marks)
        schur_fails, schur_values = schur_test(B_proof)
        bourbaki_green_fails, bourbaki_green_values = green_test(A, marks)
        item = {
            "type": name,
            "rank": len(A),
            "root_count": count,
            "expected_root_count": expected_counts[name],
            "highest_root_marks": marks,
            "green_ratio_failures_proof_convention": green_fails,
            "green_ratio_values_proof_convention": green_values,
            "schur_failures_proof_convention": schur_fails,
            "green_ratio_failures_bourbaki_orientation": len(bourbaki_green_fails),
            "green_ratio_values_bourbaki_orientation": bourbaki_green_values,
            "schur_values_proof_convention": schur_values,
        }
        summary.append(item)
        if count != expected_counts[name] or green_fails or schur_fails:
            failures.append(item)

    report = {
        "truth_label": "FINITE_CARTAN_EXACT_SAGE_TEST",
        "manuscript_version": "v1.3 full-audit package",
        "receipt_level": "full detailed per-type audit",
        "scope": "A1-A8, B2-B8, C2-C8, D4-D8, E6, E7, E8, F4, G2",
        "important_convention": "Green ratio lemma should be stated using B = transpose(Bourbaki Cartan), i.e.
             B_ij=(beta_i,beta_j^vee). It fails in non-simply-laced types if the opposite convention is used.",
```

```
        "tested_claims": [
            "root counts match expected finite root systems",
            "Green ratio maximum lemma in proof Cartan convention",
            "Schur complement bound a^T B^-1 b < 2 for every deleted node",
        ],
        "not_tested": [
            "Cellini-Marietti c_epsilon exact notation bridge",
            "external referee acceptance",
            "full OPAC-018 transfer beyond the cited reduction",
        ],
        "total_types": len(summary),
        "failures": failures,
        "passed": len(failures) == 0,
        "summary": summary,
    }
    os.makedirs("receipts", exist_ok=True)
    with open("receipts/opac18_green_schur_sage_results.json", "w") as f:
        json.dump(json_safe(report), f, indent=2, sort_keys=True)
    print(json.dumps(json_safe({"passed": report["passed"], "total_types": report["total_types"], "failures":
         len(report["failures"]), "output": "receipts/opac18_green_schur_sage_results.json"}), indent=2))


main()
```

Appendix G

Sage JSON Output Summary

The repository package contains the full detailed per-type audit receipt at

`receipts/opac18_green_schur_sage_results.json`

The file is not only a summary: it records all 32 tested finite Cartan systems, highest-root marks, Green-ratio values, Schur complement values, and failure lists. A compact preview is:


```
{
  "passed": true,
  "total_types": 32,
  "failures": [],
  "truth_label": "FINITE_CARTAN_EXACT_SAGE_TEST",
  "scope": "A1-A8, B2-B8, C2-C8, D4-D8, E6, E7, E8, F4, G2",
  "important_convention": "B_ij=(beta_i,beta_j^vee)"
}
```


The receipt is an exact-arithmetic audit asset. It is not a substitute for independent referee verification of the manuscript proof.

Appendix H

Boundary Cases and Degeneracy Notes

Rank-one and zero-adjacency cases are harmless: if there are no adjacent components contributing to r_α, then $r_\alpha = 0 < 2$. Disconnected B is handled by block decomposition and nonnegative inverse-Cartan entries on each finite-type component. Non-simply-laced systems are handled by the explicit orientation convention. Strictness follows from Schur complement positivity.

Appendix I

Repository Upload Order

Suggested public repository name:

`AXEZENT-AI-OPAC018-Green-Schur-Bridge`

Suggested layout:

```
paper/main.tex
paper/Green-Schur-Bridge-for-OPAC018-Complete-Referee-Edition-v1_4.pdf
sage/opac18_green_schur_sage_test_fixed.sage
receipts/opac18_green_schur_sage_results.json
receipts/SHA256SUMS.txt
README.md
CLAIMS_AND_NONCLAIMS.md
SUBMISSION_COVER_LETTER_DRAFT.md
verify_manifest.py
```

Appendix J

Journal Cover Letter Draft

Dear Editor-in-Chief,

I am writing to submit the manuscript titled *The Green–Schur Bridge for OPAC-018: A Classification-Free Cartan–Schur Proof Candidate of the Root Polytope Projection Bound* for consideration as a research article.

The manuscript addresses Open Problem 18 from the Open Problems in Algebraic Combinatorics collection. The problem asks for a uniform proof of the root-polytope projection dilation bound. The manuscript introduces a Green–Schur bridge: a type-free inverse-Cartan ratio inequality combined with a Schur complement argument. The proof establishes the local inequality $r_\alpha < 2$ and then applies the Cellini–Marietti reduction framework to obtain $\kappa(\Phi, U) < 2$.

Exact Sage verification assets are included as supplementary audit materials. These assets audit the index convention and finite-type Cartan arithmetic, but they are not used as a logical substitute for the proof.

This manuscript is original and is not under consideration elsewhere.

Sincerely,

Shawn Calvin Snelling
Axezent AI Research Lab

Bibliography

[1] Open Problems in Algebraic Combinatorics, “Root polytope projections,” submitted by Sam Hopkins, October 13, 2019. URL: `https://realopacblog.wordpress.com/2019/10/13/root-polytope-projections/`.

[2] P. Cellini and M. Marietti, “Root systems, affine subspaces, and projections,” *Journal of Algebra* 587 (2021), 310–335. arXiv:2109.00944. DOI: `https://doi.org/10.1016/j.jalgebra.2021.07.035`.

[3] S. Hopkins and A. Postnikov, “A positive formula for the Ehrhart-like polynomials from root system chip-firing,” *Algebraic Combinatorics* 2 (2019), no. 6, 1159–1196. DOI: `https://doi.org/10.5802/alco.79`.

[4] A. Berman and R. J. Plemmons, *Nonnegative Matrices in the Mathematical Sciences*, SIAM Classics in Applied Mathematics.

[5] R. A. Horn and C. R. Johnson, *Matrix Analysis*, Cambridge University Press.

[6] V. Kac, *Infinite-Dimensional Lie Algebras*, Cambridge University Press.

[7] J. E. Humphreys, *Introduction to Lie Algebras and Representation Theory*, Springer.